Near-real-time Solutions for Online String Problems

WAAC 2026

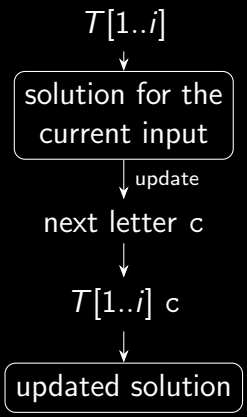
Dominik Köppl [1] Gregory Kucherov [2]

[1]: University of Yamanashi
[2]: Gustave Eiffel University

Presented at CPM'26

background

- many algorithms are offline: compute solution for the entire input
- while reading the input, we may want to know the solution for the part read so far
- i.e., at each point in time, for the text $T[1..i]$, we want to inspect a snapshot of the solution currently being maintained
- in an online algorithm, whenever a new letter arrives, the previous solution is updated efficiently



Here, an *intermediate solution* is not an approximate solution, but the *complete* solution for the prefix received so far.

online algorithms

Definition (online algorithm)

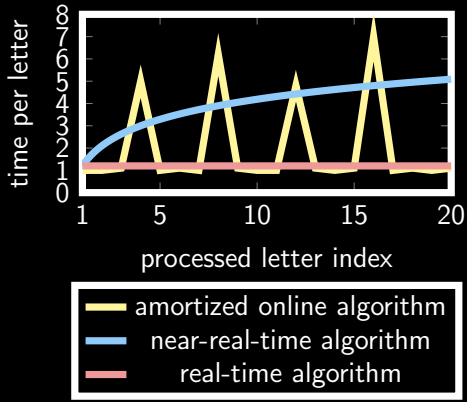
update current solution incrementally as new letters arrive

however:

- most online algorithms are amortized: worst-case time per letter may be high!
- real-time algorithm: worst-case time per letter is a *constant*

here:

- near real-time algorithm: worst-case time per letter is *poly-log-logarithmic* in the input size



real-time algorithms

What can be done in real-time?

- pattern matching with sliding window Galil'76
- palindrome recognition Galil'76
- Lempel–Ziv 78-like factorization Hartman,Rodeh'85

What can be done in near real-time?

- suffix tree construction Breslauer,Italiano'13
  based on Weiner's algorithm Weiner'73

Weiner's suffix tree construction

input: text $T[1..n]$ of length n over an integer alphabet Σ

- letters arrive from right to left
- assume $T[n] = \$$ is a unique terminal letter

output: suffix tree $\mathrm{ST}(T)$ of T

goal: construct $\mathrm{ST}(T[j..])$ from $\mathrm{ST}(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \ldots, 1$

Example
($T = \mathtt{aababaa\$}$)

Weiner's suffix tree construction

input: text $T[1..n]$ of length n over an integer alphabet Σ

- letters arrive from right to left
- assume $T[n] = \$$ is a unique terminal letter

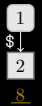
output: suffix tree $\mathrm{ST}(T)$ of T

goal: construct $\mathrm{ST}(T[j..])$ from $\mathrm{ST}(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \ldots, 1$

Example (T = aababaa\$)

- start with \$

ST(\$)



Weiner's suffix tree construction

input: text $T[1..n]$ of length n over an integer alphabet Σ

- letters arrive from right to left
- assume $T[n] = \$$ is a unique terminal letter

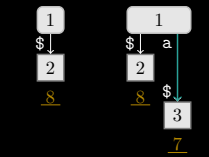
output: suffix tree $\mathrm{ST}(T)$ of T

goal: construct $\mathrm{ST}(T[j..])$ from $\mathrm{ST}(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \ldots, 1$

Example (T = aababaa$)

- start with $
- insert a$

ST($) ST(a$)



Weiner's suffix tree construction

input: text $T[1..n]$ of length n over an integer alphabet Σ

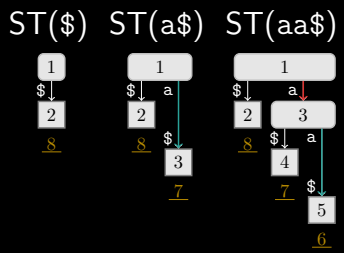
- letters arrive from right to left
- assume $T[n] = \$$ is a unique terminal letter

output: suffix tree $\mathrm{ST}(T)$ of T

goal: construct $\mathrm{ST}(T[j..])$ from $\mathrm{ST}(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \ldots, 1$

Example (T = aababaa$)

- start with $
- insert a$
- insert aa$



Weiner's suffix tree construction

input: text $T[1..n]$ of length n over an integer alphabet Σ

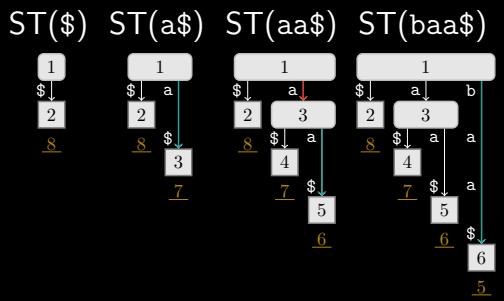
- letters arrive from right to left
- assume $T[n] = \$$ is a unique terminal letter

output: suffix tree $\mathrm{ST}(T)$ of T

goal: construct $\mathrm{ST}(T[j..])$ from $\mathrm{ST}(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \ldots, 1$

Example (T = aababaa$)

- start with $
- insert a$
- insert aa$
- insert baa$

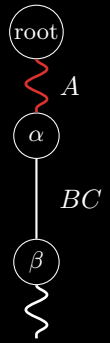


insertion point

where do we update ST when inserting $T[j..]$ into $\mathsf{ST}(T[j+1..])$?

naive solution

1. search for the longest path from the root in $\mathsf{ST}(T[j+1..])$ matching $T[j..]$

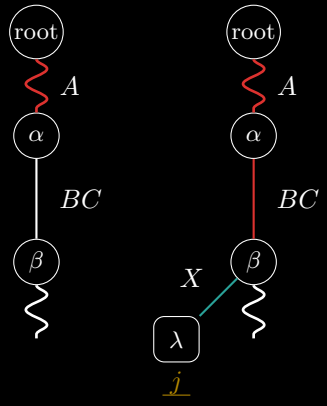


insertion point

where do we update ST when inserting $T[j..]$ into $\mathsf{ST}(T[j+1..])$?

naive solution

1. search for the longest path from the root in $\mathsf{ST}(T[j+1..])$ matching $T[j..]$
2. if $T[j..] = ABCX$, then branch off at node β



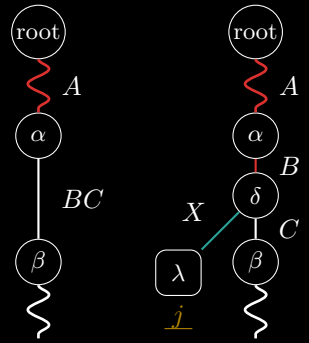
insertion point

where do we update ST when inserting $T[j..]$ into $\mathrm{ST}(T[j+1..])$?

naive solution

1. search for the longest path from the root in $\mathrm{ST}(T[j+1..])$ matching $T[j..]$
2. if $T[j..] = ABCX$, then branch off at node β
3. if $T[j..] = ABX$, then
   - 3.1 split edge (α, β) to create node δ
   - 3.2 branch off at node δ

In any case, we call the branch-off locus the insertion point.



Insertion Point Problem

Key observation

- suffix $T[j..]$ is represented by a leaf λ in $\mathsf{ST}(T[j..])$
- to update $\mathsf{ST}(T[j+1..])$ to $\mathsf{ST}(T[j..])$, we need to insert λ
- this requires finding the insertion point for $T[j..]$ in $\mathsf{ST}(T[j+1..])$

Problem (InsertionPoint)

input: suffix tree $\mathsf{ST}(T[j+1..])$ and letter $T[j]$.
output: the locus of the longest prefix of $T[j..]$ appearing in $T[j+1..]$.

Let t_{SU} denote the time complexity of one Weiner update, which includes solving InsertionPoint.

Time Complexity t_{SU}

naive approach:

- traverse $\mathsf{ST}(T[j+1..])$ from root to match $T[j..]$
- $O(n)$ time

Can we do better?

t_{SU}	Reference
$O(\lg \sigma)$ amortized	Weiner'73
$O(\lg n)$	Amir,Kopelowitz,+'05
$O(\sigma \log\log n)$	Breslauer,Italiano'13
$O(\log\log n + \log\log \sigma)$ expected	Kopelowitz'12
$O(\log\log n + \frac{\log^2 \log \sigma}{\log\log\log \sigma})$	Fischer,Gawrychowski'15
$O(\log\log n)$ for $\sigma = O(\log^{1/4} n)$	Kucherov,Nekrich'17

About Insertion Point

What does the insertion point encode?

- the longest common prefix between $T[j..n]$ and the suffix in $\mathsf{ST}(T[j+1..n])$ that is *lexicographically closest* to $T[j..n]$ (i.e., lexicographic predecessor or successor)

Can solve longest repeating prefix problem!

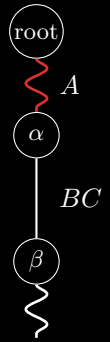
Problem (LongestRepeatingPrefix)

Input: suffix tree $\mathsf{ST}(T[j+1..])$ and letter $T[j]$.
Output: length of the longest repeating prefix of $T[j..]$.

solving LongestRepeatingPrefix via InsertionPoint

algorithm

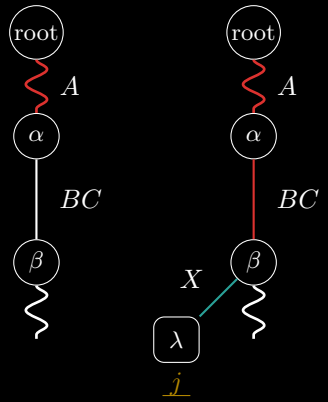
1. Compute InsertionPoint for $T[j..]$ in $\mathsf{ST}(T[j+1..])$



solving LongestRepeatingPrefix via InsertionPoint

algorithm

1. Compute InsertionPoint for $T[j..]$ in $\mathsf{ST}(T[j+1..])$
2. If InsertionPoint is β return $|ABC|$



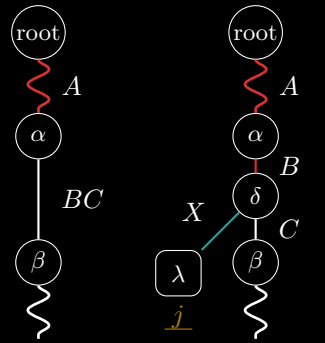
solving LongestRepeatingPrefix via InsertionPoint

algorithm

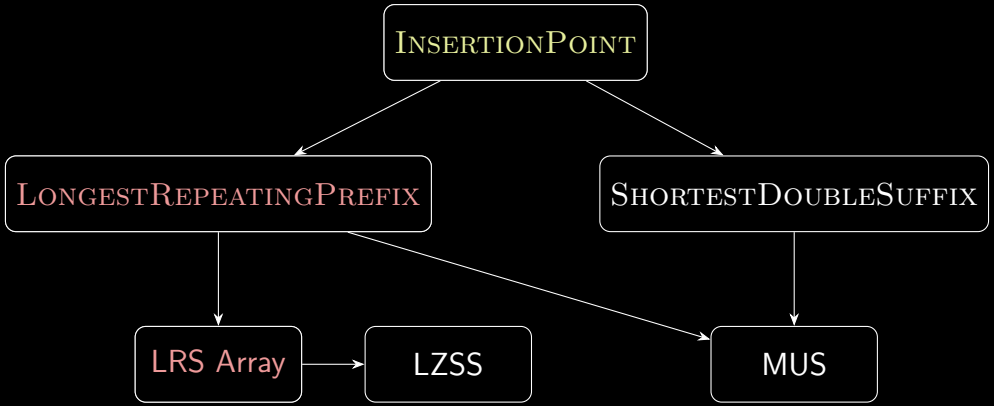
1. Compute InsertionPoint for $T[j..]$ in $\mathsf{ST}(T[j+1..])$
2. If InsertionPoint is β return $|ABC|$
3. If InsertionPoint is δ return $|AB|$

In any case: output string depth of InsertionPoint

What else can we do with it?



longest repeating suffix array



longest repeating suffix array

using LongestRepeatingPrefix we can compute:

Definition (longest repeating suffix array (LRS))

The longest repeating suffix array (LRS) of a string $T[1..n]$ is an array LRS$[1..n]$ such that for each position $j \in [1..n]$, LRS$[j]$ is the length of the longest suffix of $T[1..j]$ that occurs at least twice in $T[1..j]$.

known solutions to compute LRS online, time is per letter

time	reference
$O(\log^3 n)$	Okanohara,Sadakane'08
$O(\log^2 n)$ amortized	Prezza,Rosone'20
$O(t_{\mathsf{SU}})$	this work

Near-real-time LRS computation

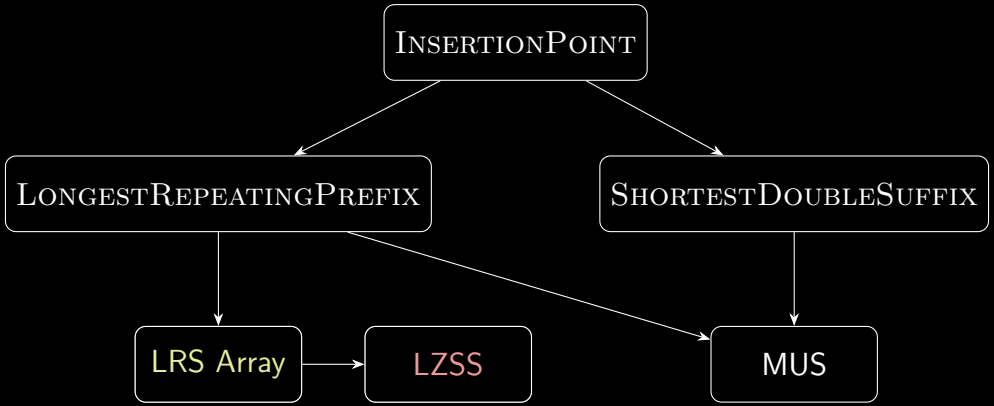
Following the ideas of Amir,Landau,+'02:

- maintain ST on reversed text $\overleftarrow{T} = T[n]T[n-1]\cdots T[1]$ online
- use LongestRepeatingPrefix queries to compute LRS

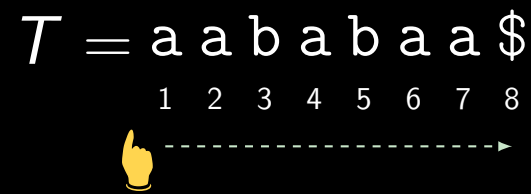
Theorem

can compute LRS online in $O(t_{\mathsf{SU}})$ worst-case time per letter.

LZSS computation



Lempel–Ziv–Storer–Szymanski (LZSS)



- encode the longest possible next substring with a back-reference

Lempel–Ziv–Storer–Szymanski (LZSS)

$$T = \mathtt{a}\,\mathtt{a}\,\mathtt{b}\,\mathtt{a}\,\mathtt{b}\,\mathtt{a}\,\mathtt{a}\,\$$$

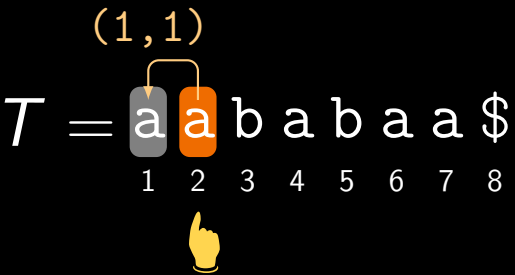
1 2 3 4 5 6 7 8



coding: a

- encode the longest possible next substring with a back-reference
- encode a letter literally if it has no previous occurrence

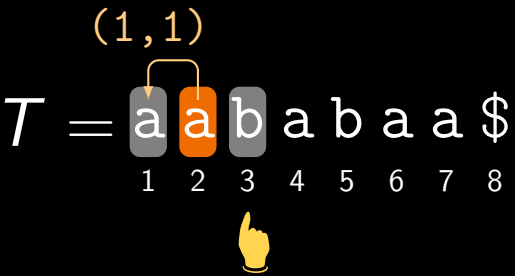
Lempel–Ziv–Storer–Szymanski (LZSS)



coding: a(1,1)

- encode the longest possible next substring with a back-reference
- encode a letter literally if it has no previous occurrence
- replace with the longest previous occurrence (overlaps are allowed)

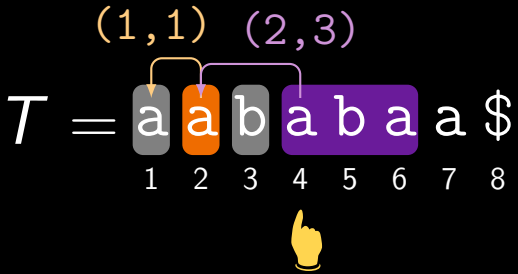
Lempel–Ziv–Storer–Szymanski (LZSS)



coding: a(1,1)b

- encode the longest possible next substring with a back-reference
- encode a letter literally if it has no previous occurrence
- replace with the longest previous occurrence (overlaps are allowed)

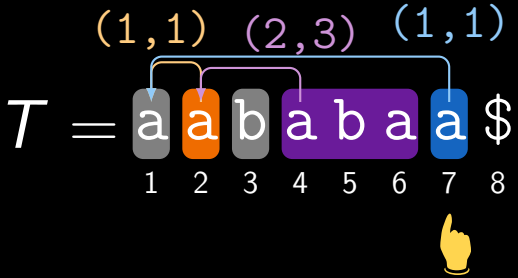
Lempel–Ziv–Storer–Szymanski (LZSS)



coding: a(1,1)b(2,3)

- encode the longest possible next substring with a back-reference
- encode a letter literally if it has no previous occurrence
- replace with the longest previous occurrence (overlaps are allowed)

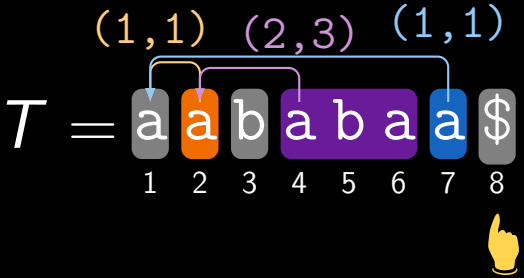
Lempel–Ziv–Storer–Szymanski (LZSS)



coding: a(1,1)b(2,3)(1,1)

- encode the longest possible next substring with a back-reference
- encode a letter literally if it has no previous occurrence
- replace with the longest previous occurrence (overlaps are allowed)

Lempel–Ziv–Storer–Szymanski (LZSS)



coding: a(1,1)b(2,3)(1,1)$

- encode the longest possible next substring with a back-reference
- encode a letter literally if it has no previous occurrence
- replace with the longest previous occurrence (overlaps are allowed)

LZSS computation

known solutions to compute LZSS online, time is per letter:

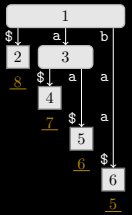
time	reference
$O(\log \sigma)$ amortized	Gusfield'97
$O(t_{\mathsf{SU}})$	this work
$O(\log^3 n)$	Okanohara,Sadakane'08
$O(\log^2 n)$ amortized	Starikovskaya'12
$O(\log n)$ amortized	Yamamoto+'14

note: latter solutions use compact space

Near-real-time LZSS computation

- suppose $j = 3$, i.e., we have processed $T[1..3] = \texttt{aab}$ of $T = \texttt{aababaa}$
- have $\mathrm{ST}(\overleftarrow{T[1..3]}\$) = \mathrm{ST}(\texttt{baa}\$)$
- know that next factor F starts at $T[4..] = \texttt{abaa}$

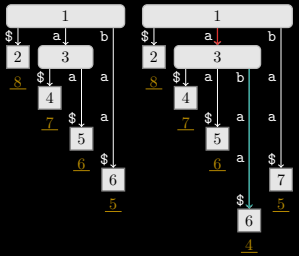
goal: compute F's length starting at $p = 4$



Near-real-time LZSS computation

goal: compute F's length starting at $p = 4$

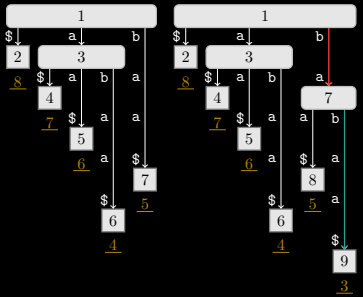
- update $j \leftarrow 4$ to $\mathsf{ST}(\overleftarrow{T[1..j]}\$) = \mathsf{ST}(\mathtt{abaa}\$)$
- insertion point has locus $L = \mathtt{a}$
- since $|L| \geq j + 1 - p = 1$: continue



Near-real-time LZSS computation

goal: compute F's length starting at $p = 4$

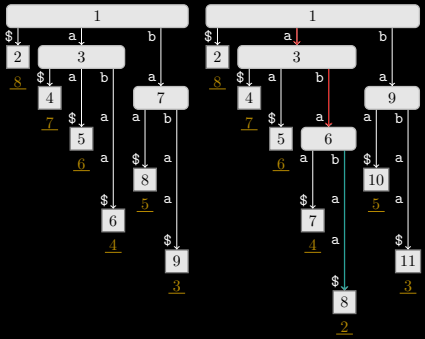
- update $j \leftarrow 5$ to $\mathrm{ST}(\overleftarrow{T[1..j]}\$) = \mathrm{ST}(\mathtt{babaa}\$)$
- insertion point has locus $L = \mathtt{ba}$
- since $|L| \geq j + 1 - p = 2$: continue



Near-real-time LZSS computation

goal: compute F's length starting at $p = 4$

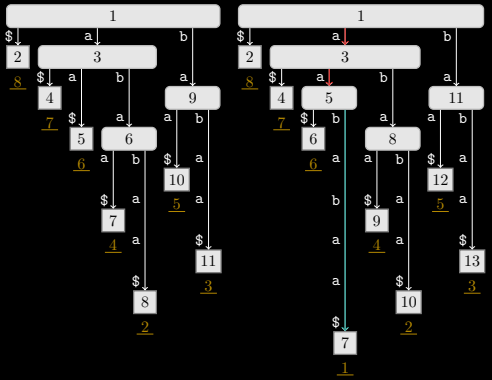
- update $j \leftarrow 6$ to $\mathsf{ST}(\overleftarrow{T[1..j]}\$) = \mathsf{ST}(\mathtt{ababaa\$})$
- insertion point has locus $L = \mathtt{aba}$
- since $|L| \geq j + 1 - p = 3$: continue



Near-real-time LZSS computation

goal: compute F's length starting at $p = 4$

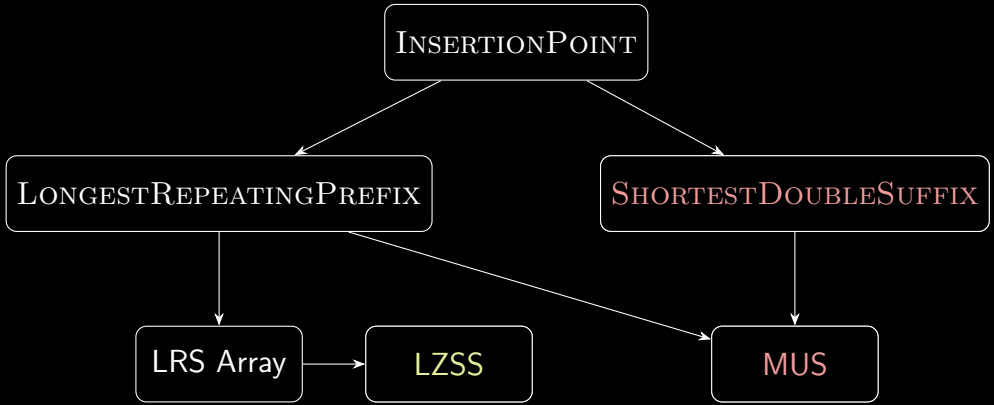
- update $j \leftarrow 7$ to $\mathrm{ST}(\overleftarrow{T[1..j]}\$) = \mathrm{ST}(\mathtt{aababaa\$})$
- insertion point has locus $L = \mathtt{aa}$
- since $|L| < j + 1 - p = 4$: stop
- know now: $|F| = j - p = 3$



LZSS computation (cont.)

Theorem

can compute LZSS factorization of T online in $O(t_{\mathsf{SU}})$ worst-case time per letter.



minimal unique substring (MUS)

another application:

Definition (minimal unique substring (MUS))

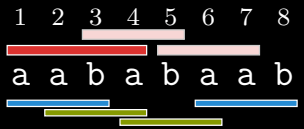
of a string $T[1..n]$ is a substring $T[\ell..r]$ such that

- $T[\ell..r]$ occurs exactly once in T
- $T[\ell+1..r]$ and $T[\ell..r-1]$ occur at least twice in T (if $\ell < r$)

Let MUS($T[1..j]$) denote the set of all MUSs of $T[1..j]$.

Example (aaba is a MUS of T)

- $T[1..4] =$ aaba occurs once
- $T[1..3] =$ aab occurs twice
- $T[2..4] =$ aba occurs twice



MUS(T) = {aaba, bab, baa}

existing work

- known solutions to compute MUS online, time is per letter

time	reference
$O(\lg \sigma)$ amortized	Mieno+'20
$O(t_{\mathsf{SU}})$	this work

growable array

is an integer array $A[1..n]$ of n elements supporting

- $A.\mathrm{grow}()$: increment the size of A
- $A[j]$: random read/write access to entry $A[j]$

growable arrays:

time	reference
$O(1)$ amortized	array doubling
$O(1)$	Brodnik+'99

approach of Mieno+'20

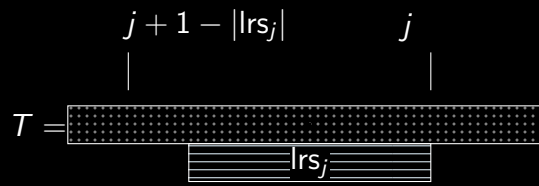
showed that MUS($T[1..j]$) can be computed from MUS($T[1..j-1]$), lrs_j and sds_j

Definition (shortest double suffix sds_j)

is the shortest suffix of $T[1..j]$ with exactly two occurrences in $T[1..j]$, which may not exist.

online MUS algorithm

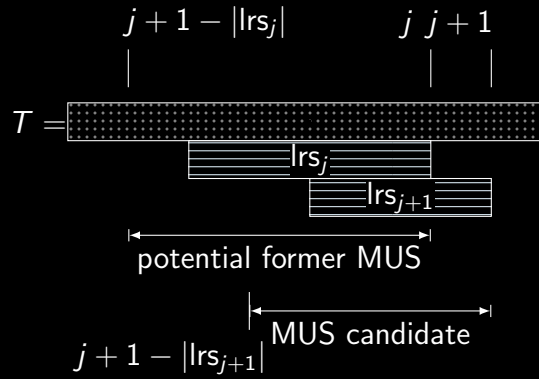
1. read new letter $T[j+1]$



online MUS algorithm

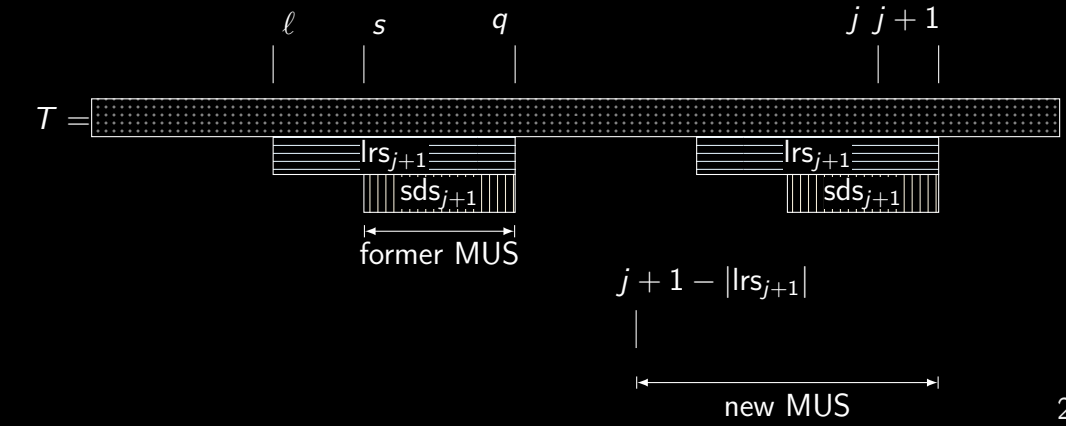
1. read new letter $T[j+1]$
2. add new MUS $[j+1-|\mathsf{lrs}_{j+1}|..j+1]$ if $|\mathsf{lrs}_{j+1}| \leq |\mathsf{lrs}_j|$

- if sds_{j+1} does not exist, we are done
- from now on: assume that sds_{j+1} exists



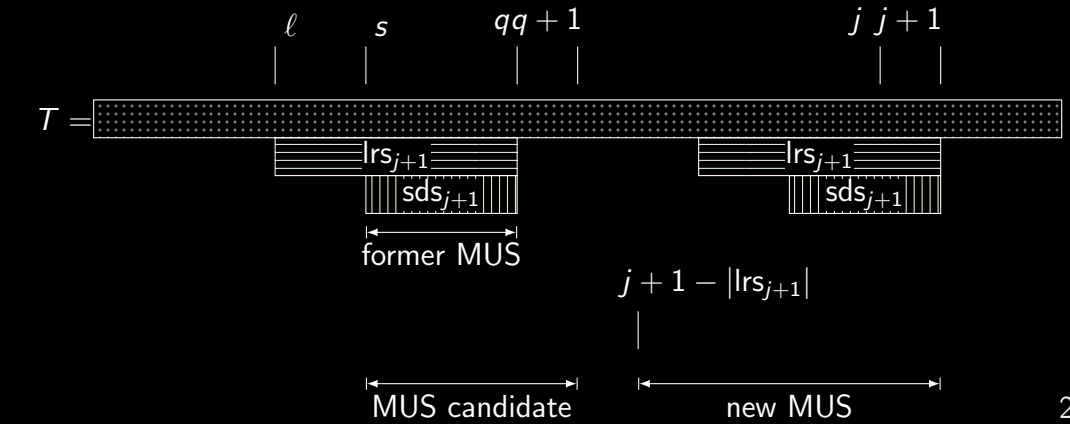
case: sds_{j+1} exists

- find sds_{j+1}'s non-suffix occurrence $[s..q]$:
  - s: the label of the leaf of the locus of sds_{j+1} that already existed in $\mathsf{ST}(\overleftarrow{T[1..j]})$
  - $q = s + |\mathsf{sds}_{j+1}| - 1$



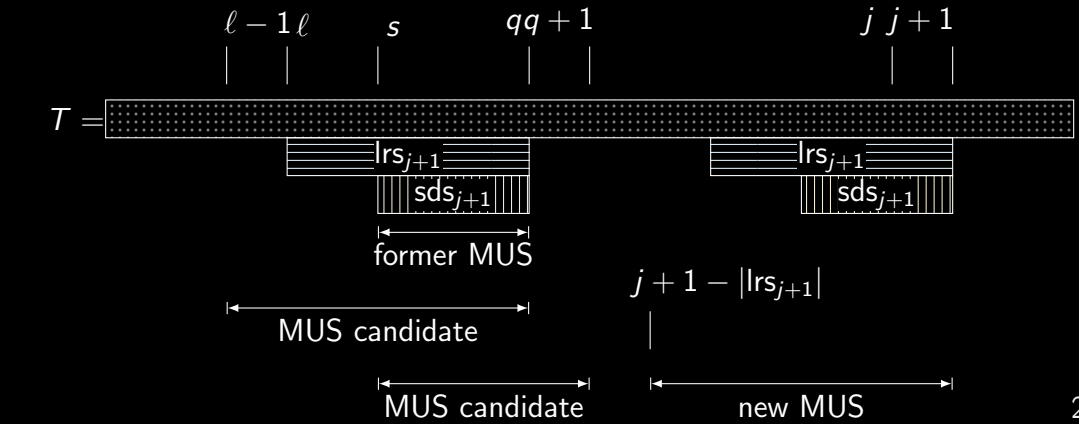
case: sds_{j+1} exists

- delete former MUS $[s..q]$
- if there is no MUS ending at position $q+1$: add MUS $[s..q+1]$



case: sds_{j+1} exists

- delete former MUS $[s..q]$
- if there is no MUS ending at position $q+1$: add MUS $[s..q+1]$
- if $q - |\mathsf{lrs}_{j+1}| \geq 1$ and $\nexists$ MUS starting at position $q - |\mathsf{lrs}_{j+1}|$: add MUS $[q - |\mathsf{lrs}_{j+1}|..q]$



reduction from InsertionPoint

Problem (ShortestDoubleSuffix sds_j)

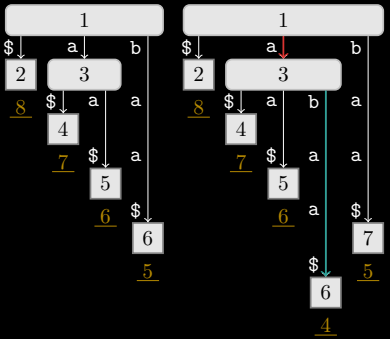
Input: suffix tree $\mathsf{ST}(\overleftarrow{T[..j-1]}\$)$ and letter $T[j]$.
Output: $\overleftarrow{\mathsf{sds}_j}$, i.e., the shortest prefix of $\overleftarrow{T[..j]}\$$ that has exactly one occurrence in $\overleftarrow{T[..j-1]}$, if it exists.

can solve ShortestDoubleSuffix via InsertionPoint!

reduction from InsertionPoint

algorithm

if the insertion point is an already existing node in ST($\overleftarrow{T[..j-1]}$\$): LongestRepeatingPrefix occurs at least three times in $T[..j]$ and sds_j cannot exist



- $T = \texttt{aababaa}$
- update from ST(baa\$) to ST(abaa\$)

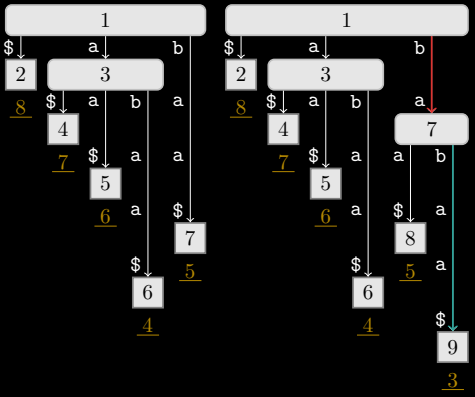
reduction from InsertionPoint

algorithm

else if LongestRepeatingPrefix is on a leaf edge:

- LongestRepeatingPrefix becomes a node in ST($\overleftarrow{T[..j]}$\$) with exactly two children
- sds_j exists and its locus is located on the parent edge to the locus of LongestRepeatingPrefix

else: the resulting locus has at least three leaves in its subtree, so sds_j cannot exist



- T = aababaa
- update from ST(abaa\$) to ST(babaa\$)

conclusion

Theorem

can compute MUS($T[1..j]$) online in $O(t_{\mathsf{SU}})$ worst-case time per letter.

summary

- real-time ST construction remains open, but near-real-time construction is possible
- many applications (LRS, LZSS, MUS, ...) solvable in near-real-time via InsertionPoint
- not shown: near-real-time computation of reversed LZ factorization / suffixient sets

Any questions are welcome!